# Shell Sort

Ordenamiento Shell: explicación detallada del algoritmo y de su visualización con bolos en Graph Maker

## 1. ¿Qué es y qué hace?

**Shell Sort** (ordenamiento Shell, inventado por Donald Shell en 1959) es una mejora de Insertion Sort. El problema de Insertion Sort es que un elemento pequeño que está muy a la derecha debe retroceder *de a una posición*. Shell Sort resuelve esto comparando e intercambiando elementos que están **separados por una distancia (gap)**.

Al principio el gap es grande, y así los elementos se acercan rápidamente a su zona correcta. Luego el gap se va reduciendo, y en la última pasada el gap vale 1, que es exactamente un Insertion Sort normal, pero sobre un arreglo que ya está casi ordenado, así que termina muy rápido.

### Analogía

Es como acomodar libros en un estante: primero mueves los libros muy desubicados a grandes saltos para dejarlos cerca de su sección correcta, y al final haces los ajustes finos de libro en libro.

## 2. Funcionamiento paso a paso

- Se elige el gap inicial: **n / 2** (división entera). En la app, la secuencia es n/2, n/4, ..., 2, 1.
- Para cada gap, se hace un Insertion Sort pero comparando elementos a distancia gap: el elemento i se compara con el de la posición i - gap.
- Si el elemento de i - gap es mayor, se intercambian y el elemento retrocede gap posiciones. Se repite mientras k >= gap.
- Cuando termina la pasada, el gap se divide entre 2.
- La última pasada con gap = 1 deja el arreglo completamente ordenado.

> Después de cada pasada con un gap dado, el arreglo queda "gap-ordenado": si se toman los elementos separados por gap posiciones, esa subsecuencia está ordenada.

## 3. Ejemplo resuelto

Se ordena el arreglo **[5, 3, 8, 1, 4]** con n = 5. Los gaps serán 5/2 = 2 y luego 1.

| Gap / i | Qué ocurre | Arreglo resultante |
|---|---|---|
| Inicio | Arreglo original. | [5, 3, 8, 1, 4] |
| gap = 2, i = 2 | Se compara 8 con el de la posición 0 (5). 5 > 8 es falso. | [5, 3, 8, 1, 4] |

| Gap / i | Qué ocurre | Arreglo resultante |
|---|---|---|
| gap = 2, i = 3 | Se compara 1 con el de la posición 1 (3). 3 > 1: se intercambian. | [5, **1**, 8, **3**, 4] |
| gap = 2, i = 4 | Se compara 4 con el de la posición 2 (8): se intercambian. Luego con el de la posición 0 (5): se intercambian. | [**4**, 1, **5**, 3, **8**] |
| gap = 1, i = 1 | Se compara 1 con 4: se intercambian. | [1, 4, 5, 3, 8] |
| gap = 1, i = 2 | Se compara 5 con 4. No se mueve. | [1, 4, 5, 3, 8] |
| gap = 1, i = 3 | El 3 se intercambia con 5 y con 4. Al compararse con 1 se detiene. | [1, 3, 4, 5, 8] |
| gap = 1, i = 4 | Se compara 8 con 5. No se mueve. | [1, 3, 4, 5, 8] |

**Observa:** con solo 5 elementos la ventaja de Shell Sort frente a Insertion Sort no se nota (en este ejemplo ambos hacen 6 intercambios). La ventaja aparece con arreglos grandes, donde los saltos grandes evitan que los elementos tengan que retroceder de uno en uno.

## Cómo se visualiza en la aplicación

En la pantalla cada número del arreglo se dibuja como un **bolo de boliche**. El tamaño del bolo es proporcional a su valor: el bolo más pequeño representa el valor mínimo y el más grande el valor máximo. El número aparece dentro del bolo. Se pueden usar entre 1 y 20 bolos, con valores entre 1 y 99.

Los controles de la pantalla son los mismos en los cuatro algoritmos:

- **Slider (barra superior):** elige la cantidad de bolos (1 a 20). Genera la secuencia 1, 2, 3, ... n. Se deshabilita mientras se ordena.
- **Cronómetro:** muestra el tiempo en formato mm:ss.d. Se actualiza cada 100 ms con un Timer y se detiene cuando termina el ordenamiento.
- **Mezclar:** desordena los valores actuales con shuffle().
- **Aleatorio:** genera valores aleatorios entre 1 y 99 (pueden repetirse).
- **Ingresar:** permite escribir los valores a mano, separados por coma o espacio, con validación (solo números, de 1 a 99, máximo 20 valores).
- **Guardar / Cargar:** guarda y recupera listas de valores. El grupo de guardado es compartido, así que lo que se guarda en un algoritmo se puede cargar en los otros.
- **Resolver:** inicia el algoritmo con animación.
- **Velocidad:** el desplegable multiplica las pausas de la animación (y la duración con la que los bolos suben y bajan). Se puede cambiar incluso mientras se ordena.

### Variables de estado que usa la animación

- **_actual:** el bolo que se está moviendo (retrocede gap posiciones).

- **_comparado:** el bolo que está a distancia gap, con el que se compara.
- **_fijos:** permanece en 0 durante todo el proceso; al final vale n y todos se pintan de verde.
- **_ordenando:** indica si el algoritmo está corriendo.

### Colores en la pantalla Shell Sort

| Color del bolo | Significado en la pantalla |
|---|---|
| Naranja | El bolo que se está moviendo (_actual). |
| Amarillo | El bolo con el que se compara, a distancia gap (_comparado). |
| Blanco | Los demás bolos durante el proceso. |
| Verde claro | Todos los bolos, únicamente al terminar. |

A diferencia de Selection e Insertion Sort, en Shell Sort no existe una zona ordenada visible a la izquierda mientras corre el algoritmo, porque los intercambios ocurren entre posiciones lejanas. Por eso el verde solo aparece al final. Las pausas son de 120 ms al comparar y 350 ms al intercambiar (por el factor de velocidad).

Si ves que los dos bolos comparados están separados varias posiciones, ¡es correcto! Es justamente el gap en acción.

## 4. Código clave

```
for (int gap = n ~/ 2; gap > 0; gap ~/= 2) {
  for (int i = gap; i < n; i++) {
    int k = i;

    while (k >= gap) {
      if (_valores[k - gap] > _valores[k]) {
        final temp = _valores[k - gap];     // intercambio a distancia gap
        _valores[k - gap] = _valores[k];
        _valores[k] = temp;
        k -= gap;                           // retrocede gap posiciones
      } else {
        break;
      }
    }
  }
}
```

En Dart, `~/` es la división entera. El ciclo externo recorre los gaps y los dos ciclos internos son un Insertion Sort con saltos de tamaño gap.

## 5. Complejidad

| Característica | Valor | Explicación |
|---|---|---|
| Mejor caso | O(n log n) | Con el arreglo ya ordenado, cada pasada solo hace comparaciones. |

| Característica | Valor | Explicación |
|---|---|---|
| Caso promedio | Depende del gap | Con esta secuencia suele comportarse mejor que $O(n^2)$ en la práctica; otras secuencias de gaps dan mejores garantías. |
| Peor caso | $O(n^2)$ | Con la secuencia n/2, n/4, ..., 1 el peor caso sigue siendo cuadrático. |
| Memoria extra | O(1) | Ordena en el mismo arreglo (in-place). |
| ¿Estable? | No | Los intercambios a distancia pueden cambiar el orden relativo de elementos iguales. |

## 6. Ventajas y desventajas

### Ventajas

- Es mucho más rápido que Insertion Sort y Selection Sort con arreglos medianos o grandes.
- No usa memoria adicional.
- Es sencillo: es un Insertion Sort con un ciclo extra para los gaps.
- Funciona bien con datos de tamaño moderado.

### Desventajas

- No es estable.
- Su rendimiento depende de la secuencia de gaps elegida; es difícil de analizar.
- Para arreglos muy grandes, Merge Sort y otros algoritmos O(n log n) son más seguros.

## 7. ¿Cuándo usarlo?

Es una buena opción cuando se quiere un algoritmo simple, sin recursión ni memoria extra, pero más rápido que los algoritmos cuadráticos básicos. Se usa en sistemas embebidos o donde no se desea el uso de memoria adicional.